

HelixTrack

HelixTrack

JIRAに代わる自由世界のための選択肢

概要

HelixTrackは、JIRA（およびDocuments拡張機能を通じたConfluence）に代わる、包括的でモダンなオープンソースの代替ソリューションです。マルチプラットフォーム対応のプロジェクト管理・課題追跡システムで、Goマイクロサービスを基盤としたバックエンドに、Web、デスクトップ、モバイル向けのネイティブクライアントを備えています。

短い説明

オープンソースのJIRA/Confluence代替システム。Goマイクロサービスバックエンド（「HelixTrack Core」）は、プロジェクト管理と課題追跡のための統合REST APIインターフェースと、Confluence風のドキュメントワークスペースを提供し、HTTP/3 QUICを介してWeb、デスクトップ、Android、iOS向けのネイティブクライアントに配信します。

長い説明

HelixTrackは、プロジェクト管理と課題追跡のためのオープンソースプラットフォームであり、エンジニアリング組織の多くが依存しているJIRAとConfluenceに代わる、自由世界のための選択肢として位置づけられています。これら2つの製品を完全に置き換えるものであり、所有権を持ち、どこでも実行可能なソフトウェアとして再構築されています。その核となるのは**HelixTrack Core**で、REST APIマイクロサービスとしてGoで実装され、Ginフレームワークを使用しています。課題追跡、アジャイル/スクラムボード、チーム管理、階層型権限エンジンを提供し、その実装はローカルのインプロセスエンジンとHTTPバックエンドサービスの間で切り替え可能です。そのため、同じ認可モデルが、単一のラップトップから分散クラスターまで、アプリケーションコードを変更することなくスケールします。Coreは、RESTのエンドポイントを多数のルートに分散させるのではなく、すべてを単一のアクションルーティングされた/doエンドポイントに

集約し、統一されたリクエスト/レスポンスのエンベロープ (action/jwt/object/dataを入力、errorCode/errorMessage/dataを出力) を採用しています。すべてのクライアントは同じ小さな契約に基づいて通信し、新しい機能の追加は新しいアクションの追加で済み、新たなURLを文書化、セキュリティ確保、バージョン管理する必要はありません。

Coreは、認証、権限、ローカライズの各サービスと連携し、これらはHTTP/3 QUICを介して通信します。これらのサービスは別々のマシンやクラスターで実行可能で、テスト環境では完全に無効化することもできます。データは、開発時にはSQLite、本番環境ではPostgreSQLに保存され、SQLCipher (AES-256) による保存時暗号化が標準で適用されるため、機密性の高いプロジェクトデータもディスク上で保護されます。これは後付けではなく、デフォルトの仕様です。**Documents V2**拡張機能により、課題追跡システムは完全なナレッジプラットフォームへと進化します。Confluence風のワークスペースとして、スペース、ページ、バージョン管理、テンプレート、リアルタイムWebSocketコラボレーション、分析機能を備え、ついにWikiと課題追跡システムが1つのバックエンドで統合され、これまでのように2つの製品を組み合わせる必要がなくなります。

Coreの周囲には、複数のクライアントアプリケーションが存在します。Angular Webクライアント、Tauri + Angularデスクトップクライアント、ネイティブのAndroid (Kotlin) およびiOS (Swift) アプリに加え、HarmonyOSやAurora OS向けのクライアント、さらにはスクリーンセーバーも提供されています。これらすべてのクライアントは同じバックエンドと通信し、UDPブロードキャストを通じてローカルネットワーク上で自動的にサーバーを検出するため、新しいクライアントでも手動設定なしでサーバーに接続できます。クライアントアプリケーションは個別の非公開リポジトリで管理されており、ここでは製品レベルでの紹介に留めています。

コンテンツ

なぜ開発したのか

チームに、JIRA + Confluenceのスタックに代わる、真にオープンで自己ホスティング可能な代替ソリューションを提供するため——「自由な世界のために」——ベンダーロックインなしで、エンタープライズグレードのトラッキング、ドキュメント、コラボレーションを一つのオープンソースライセンスのもとに統合する。

なぜゲームチェンジャーなのか

課題管理とWiki/ドキュメントスタックという二つの重量級商用製品を、一つのオープンで高性能、自己ホスティング可能なプラットフォームに統合し、さらに既存の競合製品が提供してこなかったものを組み合わせた：真のマルチプラットフォーム対応のネイティブクライアント (Web、デスクトップ、Android、iOS、さらにHarmonyOSとAurora OS) を、単一のバックエンド契約で駆動。解放されるのは、妥協なき所有権だ。HTTP/3のどこでも利用可能な完全に分離されたマイクロサービス設計と、保存時の

SQLCipher AES-256暗号化により、プロプライエタリなSaaSにのみ許されていたパフォーマンスとセキュリティ体制を、自らホスティングするシステムで実現——ライセンス数の制限も、ベンダーロックインも、データがインフラ外に流出することもない。チームは、JIRA + Confluenceの使い慣れた体験を、自社のハードウェア上で、一つのオープンソースライセンスのもとに享受できる。

革新的なポイント

- 統合アクションベースの/`do` API——一つのエンドポイント、一つのエンベロープ、アクションルーティング。新機能は新たなURLではなく新たなアクションとして追加され、攻撃対象領域、クライアントコード、ドキュメントの負担を単一の契約に集約。すべてのプラットフォームが共有する。
- HTTP/3 QUICをデフォルトのサービス間通信手段として採用——後から追加するのではなく、初日から低遅延で接続耐性のある近代的なネットワークキングを実現。
- 置き換え可能な権限エンジン——ローカルのインプロセス実装とHTTPバックエンドサービスの切り替えが可能で、オプションとして独立デプロイ可能なAuth、Permissions、Localizationサービスを併用。単一プロセスでもクラスターでも、同じ認可モデルを維持。
- `--space-root`フラグによるマルチスペースデータ分離——各プロジェクトは独立したデータベースとアセットストアを持ち、クエリフィルターではなくストレージ境界でテナントやプロジェクトを分離。
- 保存時のSQLCipher AES-256暗号化——機密性の高いプロジェクトデータは、デフォルトで透過的にディスク上で保護される。
- UDPブロードキャストによるローカルネットワーク上での自動クライアント・ツー・サーバー検出——クライアントはCoreを手動設定なしで発見。
- Documents V2——真の「Confluence代替」として、オプティミスティックロックによる並列編集、競合検出、完全な変更履歴を備えた本格的なコラボレーション文書。トラッカーと同じバックエンドで稼働。

最大の技術的課題とその解決策

- 6つのクライアントプラットフォーム、1つのバックエンド、契約のずれなし。Web/Angular、デスクトップ/Tauri、Android/Kotlin、iOS/Swift、HarmonyOS、Auroraの各クライアントを維持するには、通常6つの異なるAPI統合が必要となり、同期が崩れるリスクが生じる。このリスクを回避するため、単一のアクションルーティングされた/`do` APIとその固定エンベロープを唯一の契約とし、すべてのクライアントが同一のターゲットを持つように設計。さらにUDPブロードキャストによるサービス検出を重ね、クライアントがネットワーク上のCoreを手動設定なしで発見できるようにした。
- サービスの分離による遅延コストの回避。Auth、Permissions、Localizationを独立デプロイ可能なサービスに分割すると、通常は呼び出しごとにネットワークホップが増える。そこで、すべてのサービス間通信にHTTP/3 QUICを採用し、ホップを高速かつ接続耐性のあるもの

に維持。さらに、各サービスは独立して実行可能——テスト設定では完全に無効化も可能——にし、分離はデプロイの選択肢であって固定コストではないようにした。

- **Confluence**レベルのコラボレーションを、書き込みの競合なしで実現。リアルタイムの複数著者による編集では、書き込みの競合が発生しやすい。Documents V2は、スペース/ページ/バージョンングをオプティミスティックロックのもとに置き、明示的な競合検出と完全な変更履歴、リアルタイムのWebSocket同期を組み合わせることで、編集が密かに上書きされることなく、一貫性を保つコラボレーションを実現。
- 保存時の暗号化によるスループット低下の回避。SQLCipher
AES-256はディスク上のデータを保護するが、クエリごとにオーバーヘッドが発生する。これを、Localizationサービス内のRedisを前段に配置したメモリ内LRUキャッシュで補い、多言語検索などのホットパスは高速を維持しつつ、データは暗号化されたまま保持。

テックスタック

- **Go + Gin** — 高スループットかつ低レイテンシのHTTPサービスに最適化され、単一バイナリでのデプロイメントを実現。CoreのREST APIを搭載し、JWT/CORSミドルウェアと、システム全体を統括するアクションルーティングの/`do`ルーターを提供。
- **HTTP/3 QUIC** — Coreと認証・権限・ローカライズサービス間の通信に採用。QUICのマルチプレックスおよびコネクションマイグレーション機能により、テールレイテンシを削減し、TCPでは停止してしまう不安定な接続でも耐えうる設計。
- **PostgreSQL (本番環境) / SQLite (開発環境)** — 大規模なトラッキングとドキュメントスキーマを支える単一のリレーショナルモデル。SQLiteはローカル開発をゼロセットアップかつファイルベースで実現し、Postgresはproductionコンポーザプロファイルを通じて本番環境のスケールに対応。
- **SQLCipher (AES-256)** — データベースレベルでの透過的な保存時暗号化を採用。機密性の高いプロジェクトデータの保護にアプリケーション層の暗号化は不要で、クエリの記述方法も変更不要。
- **Redis** — ローカライズサービスのインメモリLRUキャッシュのバックエンドとして採用。二層構造のキャッシュにより、暗号化のオーバーヘッドがあっても、頻繁に使用される多言語ルックアップを高速に維持。
- **Uber Zap + Lumberjack** — 構造化された軽量なロギングと自動ローテーション機能を備え、本番環境でも無制限なログ増大を防ぎながらCoreの可観測性を確保。
- **golang-jwt / JWT** — ステートレスな認証メカニズムとして採用。署名済みトークンはすべての/`do`エンベロープのjwtフィールドに格納され、すべてのクライアントで統一された認証を実現。
- **Angular 19 (+ Material, RxJS)** — リアクティブでコンポーネント駆動のブラウザクライアントを実現。成熟したMaterialデザインシステムを標準搭載。
- **Tauri 2.0 + Rust + Angular** — RustをバックエンドとしたWebView内でAngularのUIを再利用することで、フルブラウザランタイムをバンドルせずに、軽量なネイティブデスクトップシェルを提供。

- **Kotlin (Android) / Swift + SwiftUI (iOS)** — モバイルユーザーにラップされたWebViewではなく、真にネイティブでプラットフォームに最適化されたクライアントを提供。
- **Docker / Docker Compose (Podman互換)** — 再現性の高いコンテナ化デプロイメントを実現。/healthチェックを組み込み、Podman互換性によりデーモンや特定ベンダーへの依存を排除。
- **Testify (Go) ; Cypress/Playwright/Karma+Jasmine** (クライアント) — バックエンドの契約とクライアントUIを独立してカバーする階層化された自動テストを採用。単一バックエンド・複数クライアントのアーキテクチャに対応。

ステータスと正直な注意点

- ステータス：ベータ版。 HelixTrack Coreは動作するREST APIマイクロサービスだが、**Documents V2**拡張機能は約95%完成とされており、データベースのフィールドマッピングに既知の問題が残っているため、完全リリースとはみなされていない。
- ライセンス：未定。 CLAUDE.mdではMITライセンスと記載されているが、core/LICENSEの正式ファイルはApache 2.0となっている。この不一致はライセンスの最終決定前に解消される必要がある。
- プロジェクトのREADMEに記載されているパフォーマンス数値（例：50,000リクエスト/秒以上、サブミリ秒のクエリ時間）は、設計・マーケティング上の目標値であり、独立したベンチマーク結果ではない。そのため、上記の記述からは省略している。
- クライアントアプリケーション（Web、デスクトップ、Android、iOS、Aurora、HarmonyOS）は非公開リポジトリに存在し、製品レベルでの説明に留まっている。

優先順位階層： Helix-primary、およびHelix-Track製品ラインの主力プロジェクト——Server Factoryプロジェクトに優先して位置付けられる。